OP-TEE 개발자 가이드

1. 개발 환경 설정

1.1 필수 도구 설치

```
# Ubuntu/Debian 기준
sudo apt-get update
sudo apt-get install -y \
build-essential \
gcc-aarch64-linux-gnu \
gcc-arm-linux-gnueabihf \
python3 \
python3-pip \
device-tree-compiler \
uuid-dev \
libssl-dev \
flex \
bison

# 크로스 컴파일러 설정
export CROSS_COMPILE_core=aarch64-linux-gnu-
```

```
export CROSS_COMPILE_ta_arm64=aarch64-linux-gnu-
export CROSS_COMPILE_ta_arm32=arm-linux-gnueabihf-
```

1.2 소스 코드 빌드

```
# 기본 빌드 (QEMU ARMv8-A)
make PLATFORM=vexpress-qemu_armv8a

# 특정 플랫폼 빌드 (예: i.MX8)
make PLATFORM=imx-mx8qxpmek

# 디버그 빌드
make PLATFORM=vexpress-qemu_armv8a CFG_TEE_CORE_DEBUG=y

# 병렬 빌드
make -j$(nproc) PLATFORM=vexpress-qemu_armv8a
```

1.3 주요 빌드 옵션

```
# 아키텍처 설정
ARCH=arm # ARM 32-bit
ARCH=arm64 # ARM 64-bit
```

```
# 플랫폼 설정
PLATFORM=vexpress-qemu_armv8a # QEMU ARMv8-A
PLATFORM=imx-mx8qxpmek # NXP i.MX8
PLATFORM=stm32mp1 # STM32MP1
PLATFORM=rpi3 # Raspberry Pi 3

# 디버그 옵션
CFG_TEE_CORE_DEBUG=y # 코어 디버그 활성화
CFG_TEE_CORE_LOG_LEVEL=4 # 로그 레벨 (0-4)
CFG_TEE_TA_LOG_LEVEL=4 # TA 로그 레벨

# 보안 옵션
CFG_CORE_SANITIZE_KADDRESS=y # 커널 주소 새니타이저
CFG_CORE_SANITIZE_UNDEFINED=y # 정의되지 않은 동작 새니타이저
CFG_WITH_STACK_CANARIES=y # 스택 카나리 활성화

# 성능 옵션
CFG_WITH_PAGER=y # 페이저 활성화
CFG_CORE_DYN_SHM=y # 동적 공유 메모리
CFG_CACHE_API=y # 캐시 API 지원
```

2. Trusted Application (TA) 개발

2.1 TA 프로젝트 구조

```
my_ta/
├── Makefile # 빌드 설정
├── user_ta_header_defines.h # TA 헤더 정의
├── include/
| └── my_ta.h # TA 헤더 파일
├── src/
| └── my_ta.c # TA 구현
└── ta/
└── user_ta.mk # TA 빌드 규칙
```

2.2 TA 헤더 정의 (`user_ta_header_defines.h`)

```
#ifndef USER_TA_HEADER_DEFINES_H
#define USER_TA_HEADER_DEFINES_H

#include <my_ta.h>

/* TA UUID */
#define TA_UUID MY_TA_UUID

/* TA 속성 */
```

```
#define TA_FLAGS (TA_FLAG_MULTI_SESSION |  \
TA_FLAG_EXEC_DDR)

/* TA 스택 크기 (바이트) */
#define TA_STACK_SIZE (2  *  1024)

/* TA 데이터/힙 크기 (바이트) */
#define TA_DATA_SIZE (32  *  1024)

/* TA 버전 정보 */
#define TA_VERSION "1.0"
#define TA_DESCRIPTION "My Trusted Application"

/* 지원하는 명령어 ID */
#define TA_MY_CMD_ENCRYPT 0
#define TA_MY_CMD_DECRYPT 1
#define TA_MY_CMD_GENERATE_KEY 2

#endif /* USER_TA_HEADER_DEFINES_H */
```

2.3 TA 구현 예제 (`my_ta.c`)

```
#include  <tee_internal_api.h>
```

```
#include  <tee_internal_api_extensions.h>

#include  <string.h>

#include  <my_ta.h>



/*

* TA 생성 시 호출되는 함수

*/

TEE_Result TA_CreateEntryPoint(void)

{

DMSG("TA_CreateEntryPoint called");

return TEE_SUCCESS;

}



/*

* TA 소멸 시 호출되는 함수

*/

void  TA_DestroyEntryPoint(void)

{

DMSG("TA_DestroyEntryPoint called");

}



/*

* 세션 열기 시 호출되는 함수
```

```
*/

TEE_Result TA_OpenSessionEntryPoint(uint32_t  param_types,

TEE_Param params[4],

void **sess_ctx)

{

uint32_t exp_param_types = TEE_PARAM_TYPES(TEE_PARAM_TYPE_NONE,

TEE_PARAM_TYPE_NONE,

TEE_PARAM_TYPE_NONE,

TEE_PARAM_TYPE_NONE);


DMSG("TA_OpenSessionEntryPoint called");


/* 매개변수 타입 검증 */

if (param_types != exp_param_types)

return TEE_ERROR_BAD_PARAMETERS;


/* 세션 컨텍스트 할당 (필요한 경우) */

*sess_ctx = NULL;


return TEE_SUCCESS;

}


/*
```

```
* 세션 닫기 시 호출되는 함수

*/

void  TA_CloseSessionEntryPoint(void *sess_ctx)

{

DMSG("TA_CloseSessionEntryPoint called");

/* 세션 컨텍스트 정리 */

if (sess_ctx)

TEE_Free(sess_ctx);

}



/*

* AES 암호화 함수

*/

static TEE_Result aes_encrypt(uint32_t  param_types, TEE_Param params[4])

{

TEE_Result res = TEE_SUCCESS;

TEE_OperationHandle op = TEE_HANDLE_NULL;

TEE_ObjectHandle key = TEE_HANDLE_NULL;

uint32_t exp_param_types = TEE_PARAM_TYPES(

TEE_PARAM_TYPE_MEMREF_INPUT, // 평문

TEE_PARAM_TYPE_MEMREF_INPUT, // 키

TEE_PARAM_TYPE_MEMREF_OUTPUT, // 암호문

TEE_PARAM_TYPE_NONE);
```

```
if (param_types != exp_param_types)
return TEE_ERROR_BAD_PARAMETERS;

/* AES 키 객체 생성 */
res = TEE_AllocateTransientObject(TEE_TYPE_AES,
params[1].memref.size * 8,
&key);
if (res != TEE_SUCCESS)
goto cleanup;

/* 키 속성 설정 */
TEE_Attribute attr;
TEE_InitRefAttribute(&attr, TEE_ATTR_SECRET_VALUE,
params[1].memref.buffer,
params[1].memref.size);

res = TEE_PopulateTransientObject(key, &attr, 1);
if (res != TEE_SUCCESS)
goto cleanup;

/* 암호화 연산 할당 */
res = TEE_AllocateOperation(&op, TEE_ALG_AES_ECB_NOPAD,
```

```
TEE_MODE_ENCRYPT,

params[1].memref.size * 8);

if (res != TEE_SUCCESS)

goto cleanup;



/* 키 설정 */

res = TEE_SetOperationKey(op, key);

if (res != TEE_SUCCESS)

goto cleanup;



/* 암호화 수행 */

res = TEE_CipherDoFinal(op,

params[0].memref.buffer,

params[0].memref.size,

params[2].memref.buffer,

&params[2].memref.size);



cleanup:

if (op != TEE_HANDLE_NULL)

TEE_FreeOperation(op);

if (key != TEE_HANDLE_NULL)

TEE_FreeTransientObject(key);
```

```
return res;

}



/*

* 명령어 처리 함수

*/

TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx,

uint32_t  cmd_id,

uint32_t  param_types,

TEE_Param params[4])

{

DMSG("TA_InvokeCommandEntryPoint called with cmd_id=%u", cmd_id);



switch (cmd_id) {

case TA_MY_CMD_ENCRYPT:

return  aes_encrypt(param_types, params);

case TA_MY_CMD_DECRYPT:

/* 복호화 구현 */

return TEE_ERROR_NOT_IMPLEMENTED;

case TA_MY_CMD_GENERATE_KEY:

/* 키 생성 구현 */

return TEE_ERROR_NOT_IMPLEMENTED;

default:
```

```
return TEE_ERROR_BAD_PARAMETERS;

}

}
```

2.4 TA 빌드 설정 (Makefile)

```
# TA 개발 키트 경로

TA_DEV_KIT_DIR ?= $(OPTEE_OS_DIR)/out/arm-plat-vexpress/export-ta_arm32


# TA UUID (16진수 형태)

BINARY = 8aaaf200-2450-11e4-abe2-0002a5d5c51b


# 소스 파일들

SRCS = my_ta.c


# 라이브러리 링크

LIBNAMES += utils

LIBNAMES += utee


# 컴파일러 플래그

CFLAGS += -Wall -Wextra -Werror

CPPFLAGS += -I../include
```

```
# TA 개발 키트 포함

include  $(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk
```

3. 고급 TA 개발 기법

3.1 보안 스토리지 사용

```
/* 보안 객체 생성 및 저장 */

TEE_Result store_secure_data(const  void *data, size_t  data_len,

const  char *obj_id)

{

TEE_Result res;

TEE_ObjectHandle obj = TEE_HANDLE_NULL;

uint32_t flags = TEE_DATA_FLAG_ACCESS_WRITE |

TEE_DATA_FLAG_ACCESS_READ |

TEE_DATA_FLAG_OVERWRITE;



/* 보안 스토리지 객체 생성 */

res = TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE,

obj_id, strlen(obj_id),

flags, TEE_HANDLE_NULL,

data, data_len,

&obj);
```

```
if (res != TEE_SUCCESS)

return res;



TEE_CloseObject(obj);

return TEE_SUCCESS;

}



/* 보안 객체 읽기 */

TEE_Result read_secure_data(void *data, size_t *data_len,

const  char *obj_id)

{

TEE_Result res;

TEE_ObjectHandle obj = TEE_HANDLE_NULL;

uint32_t flags = TEE_DATA_FLAG_ACCESS_READ;



/* 보안 스토리지 객체 열기 */

res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,

obj_id, strlen(obj_id),

flags, &obj);

if (res != TEE_SUCCESS)

return res;



/* 데이터 읽기 */
```

```
res = TEE_ReadObjectData(obj, data, *data_len, data_len);

TEE_CloseObject(obj);

return res;

}
```

3.2 암호화 키 생성 및 관리

```
/* RSA 키 쌍 생성 */

TEE_Result generate_rsa_keypair(uint32_t  key_size)

{

TEE_Result res;

TEE_ObjectHandle keypair = TEE_HANDLE_NULL;

/* RSA 키 쌍 객체 할당 */

res = TEE_AllocateTransientObject(TEE_TYPE_RSA_KEYPAIR,

key_size, &keypair);

if (res != TEE_SUCCESS)

return res;


/* 키 쌍 생성 */

res = TEE_GenerateKey(keypair, key_size, NULL, 0);

if (res != TEE_SUCCESS)

goto cleanup;
```

```
/* 공개키 추출 */

TEE_Attribute attrs[3];

uint32_t attr_count = 0;

res = TEE_GetObjectBufferAttribute(keypair, TEE_ATTR_RSA_MODULUS,

NULL, &attrs[0].content.ref.length);

if (res != TEE_SUCCESS)

goto cleanup;



attrs[0].content.ref.buffer = TEE_Malloc(attrs[0].content.ref.length, 0);

if (!attrs[0].content.ref.buffer) {

res = TEE_ERROR_OUT_OF_MEMORY;

goto cleanup;

}



res = TEE_GetObjectBufferAttribute(keypair, TEE_ATTR_RSA_MODULUS,

attrs[0].content.ref.buffer,

&attrs[0].content.ref.length);

/* ... 추가 속성 처리 ... */



cleanup:

if (keypair != TEE_HANDLE_NULL)

TEE_FreeTransientObject(keypair);

return res;
```

```
}
```

3.3 시간 및 난수 서비스

```
/* 현재 시간 얻기 */
void  get_current_time(void)
{
TEE_Time time;
TEE_GetSystemTime(&time);
DMSG("Current time: %u.%03u seconds",
time.seconds, time.millis);
}

/* 안전한 난수 생성 */
TEE_Result generate_random_bytes(void *buffer, size_t  size)
{
TEE_GenerateRandom(buffer, size);
return TEE_SUCCESS;
}

/* 타이머 설정 */
TEE_Result set_timer(uint32_t  timeout_ms)
{
```

```
TEE_Time timeout;

timeout.seconds = timeout_ms / 1000;

timeout.millis = timeout_ms % 1000;

return  TEE_SetTAPersistentTime(&timeout);

}
```

4. 디버깅 및 테스트

4.1 로깅 및 추적

```
/* 로그 레벨별 출력 */

#include  <trace.h>


void  debug_function(void)

{

EMSG("Error message"); // 에러 (레벨 1)

IMSG("Info message"); // 정보 (레벨 2)

DMSG("Debug message"); // 디버그 (레벨 3)

FMSG("Flow message"); // 플로우 (레벨 4)

}


/* 조건부 로깅 */

#ifdef CFG_TEE_TA_LOG_LEVEL
```

```
#if CFG_TEE_TA_LOG_LEVEL >=  3

DMSG("This will only appear in debug builds");

#endif

#endif
```

4.2 어설션 및 에러 처리

```
#include  <assert.h>


TEE_Result secure_function(void *buffer, size_t  size)

{

/* 매개변수 검증 */

if (!buffer || size == 0)

return TEE_ERROR_BAD_PARAMETERS;

/* 크기 제한 검사 */

if (size > MAX_BUFFER_SIZE) {

EMSG("Buffer size too large: %zu", size);

return TEE_ERROR_EXCESS_DATA;

}

/* 런타임 어설션 */

assert(buffer != NULL);

/* 메모리 정렬 검사 */

if ((uintptr_t)buffer & 0x3) {
```

```
EMSG("Buffer not aligned");

return TEE_ERROR_BAD_PARAMETERS;

}

return TEE_SUCCESS;

}
```

4.3 성능 측정

```
/* 실행 시간 측정 */

void  measure_performance(void)

{

TEE_Time start, end;

uint32_t elapsed_ms;

TEE_GetSystemTime(&start);

/* 측정할 코드 */

perform_crypto_operation();

TEE_GetSystemTime(&end);

elapsed_ms = (end.seconds - start.seconds) * 1000 +

(end.millis - start.millis);

IMSG("Operation took %u ms", elapsed_ms);

}


/* 메모리 사용량 체크 */
```

```
void  check_memory_usage(void)

{

void *ptr = TEE_Malloc(1024, 0);

if (!ptr) {

EMSG("Memory allocation failed");

return;

}

/* 메모리 사용 */

memset(ptr, 0, 1024);

/* 정리 */

TEE_Free(ptr);

}
```

5. 최적화 기법

5.1 메모리 최적화

```
/* 스택 메모리 사용 최소화 */

TEE_Result process_large_data(const  void *input, size_t  input_size)

{

/* 큰 배열을 스택에 선언하지 말고 동적 할당 사용 */

void *work_buffer = TEE_Malloc(input_size, 0);

if (!work_buffer)
```

```
return TEE_ERROR_OUT_OF_MEMORY;

/* 처리 로직 */

memcpy(work_buffer, input, input_size);

/* 정리 */

TEE_Free(work_buffer);

return TEE_SUCCESS;

}



/* 메모리 풀 사용 */

#define POOL_SIZE 10

static  void *memory_pool[POOL_SIZE];

static bool pool_used[POOL_SIZE];



void* pool_alloc(size_t size)

{

if (size > POOL_BLOCK_SIZE)

return TEE_Malloc(size, 0);

for (int i = 0; i < POOL_SIZE; i++) {

if (!pool_used[i]) {

pool_used[i] = true;

return memory_pool[i];

}

}
```

```
return NULL;

}
```

5.2 암호화 최적화

```
/* 하드웨어 가속 활용 */

TEE_Result optimized_aes_encrypt(const void *plaintext, size_t pt_len,

void *ciphertext, size_t *ct_len,

const void *key, size_t key_len)

{

TEE_Result res;

TEE_OperationHandle op = TEE_HANDLE_NULL;

TEE_ObjectHandle key_obj = TEE_HANDLE_NULL;

/* 하드웨어 가속 지원 알고리즘 사용 */

uint32_t algorithm = TEE_ALG_AES_ECB_NOPAD;

/* 키 크기에 따른 최적화 */

uint32_t key_bits = key_len * 8;

if (key_bits != 128 && key_bits != 192 && key_bits != 256) {

return TEE_ERROR_BAD_PARAMETERS;

}

/* 연산 할당 및 수행 */

res = TEE_AllocateOperation(&op, algorithm, TEE_MODE_ENCRYPT, key_bits);

if (res != TEE_SUCCESS)
```

```
return res;

/* ... 나머지 구현 ... */

return res;

}
```

6. 보안 고려사항

6.1 입력 검증

```
/* 철저한 입력 검증 */

TEE_Result validate_input(const void *data, size_t size,

size_t min_size, size_t max_size)

{

/* NULL 포인터 검사 */

if (!data)

return TEE_ERROR_BAD_PARAMETERS;

/* 크기 범위 검사 */

if (size < min_size || size > max_size)

return TEE_ERROR_BAD_PARAMETERS;

/* 정수 오버플로우 검사 */

if (size > SIZE_MAX - sizeof(struct header))

return TEE_ERROR_BAD_PARAMETERS;

/* 메모리 정렬 검사 (필요한 경우) */
```

```
if ((uintptr_t)data & (ALIGNMENT - 1))

return TEE_ERROR_BAD_PARAMETERS;

return TEE_SUCCESS;

}
```

6.2 메모리 보안

```
/* 민감한 데이터 처리 */

void secure_memory_handling(void)

{

uint8_t sensitive_data[32];

/* 민감한 데이터 사용 */

TEE_GenerateRandom(sensitive_data, sizeof(sensitive_data));

/* 사용 후 즉시 지우기 */

TEE_MemFill(sensitive_data, 0, sizeof(sensitive_data));

}


/* 버퍼 오버플로우 방지 */

TEE_Result safe_copy(void *dst, size_t dst_size,

const void *src, size_t src_size)

{

if (src_size > dst_size)

return TEE_ERROR_SHORT_BUFFER;
```

```
    TEE_MemMove(dst, src, src_size);

    return TEE_SUCCESS;

}
```

이 가이드는 OP-TEE에서 Trusted Application을 개발하기 위한 실용적인 정보를 제공합니다. 보안성과 성능을 모두 고려한 안전한 TA 개발을 위해 제시된 모범 사례들을 따르시기 바랍니다.